

M09 - ETL/ELT + Data Quality + Transformation Engineering

13-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Build deterministic ETL/ELT transformations with contracts, reconciliation, tests and incremental behavior.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M08 ingestion foundations plus M02-M03 SQL. You should understand source/landing/staging boundaries, reruns, and reconciliation.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE09-01 - ETL vs ELT and transformation boundaries	5
DE09-02 - Raw, staging, clean/intermediate and serving contracts	7
DE09-03 - Deterministic transformations and production SQL style	9
DE09-04 - Deduplication and key integrity	11
DE09-05 - Schema checks and data contracts	13
DE09-06 - Source-to-target reconciliation	15
DE09-07 - Incremental processing and safe reruns	17
DE09-08 - Failure behavior, retries, quarantine and atomic state	19
DE09-09 - Lineage, run metadata and auditability	21
DE09-10 - Build and defend a reliable batch pipeline	23
DE09-11 - dbt-style project anatomy, sources and dependency graph	25
DE09-12 - Executable data tests, quality severity and CI boundaries	27
DE09-13 - Incremental models, materialization, documentation and handoff	29

Prerequisite, tool and outcome map

ASSUMED

Complete M08 ingestion foundations plus M02-M03 SQL. You should understand source/landing/staging boundaries, reruns, and reconciliation.

MODULE PURPOSE

Build deterministic ETL/ELT transformations with contracts, reconciliation, tests and incremental behavior.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE09-01 - ETL vs ELT and transformation boundaries

1. Why it matters

Choose ETL vs ELT boundaries from trust, compute and ownership rather than slogans. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Choose ETL vs ELT boundaries from trust, compute and ownership rather than slogans.

3. Prerequisite / what you already know

Complete M08 ingestion foundations plus M02-M03 SQL. You should understand source/landing/staging boundaries, reruns, and reconciliation.

4. Picture it in your head

Imagine an airport baggage flow: ETL opens, checks and repacks bags before they enter the destination store; ELT lands the bags first, then sorts and reshapes them inside. The design decision is where the inspection desk, ownership and recovery boundary should live.

5. New words before we use them

ETL transforms before final load; ELT lands source-shaped data first and transforms inside the destination. Both need contracts, lineage and reconciliation.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — Introduction to Analytics Engineering

Assigned scope: Whole introduction — ETL/ELT boundary and analytics-engineering role

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=HxMIsPrlyGQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Draw source ->

landing/raw ->

validated staging ->

intermediate ->

serving. Mark where each rule is enforced. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to choose ETL vs ELT boundaries from trust, compute and ownership rather than slogans and show a concrete result/state that another person can inspect. Observable target: For ETL vs ELT and transformation boundaries: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because choose ETL vs ELT boundaries from trust, compute and ownership rather than slogans is made explicit before the result is accepted. Your independent check is: Do not validate ETL vs ELT and transformation boundaries by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If you argue only from tool preference, restate the data-risk, recoverability and ownership requirements.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Imagine an airport baggage flow: ETL opens, checks and repacks bags before they enter the destination store; ELT lands the bags first, then sorts and reshapes them inside. 3. Run the earliest independent control: Do not validate ETL vs ELT and transformation boundaries by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Given a CSV plus a warehouse, justify which cleanup must happen before load and which business modeling can happen after load.

13. Validate your result

Do not validate ETL vs ELT and transformation boundaries by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain ETL vs ELT and transformation boundaries in your own words. Start with the purpose: Choose ETL vs ELT boundaries from trust, compute and ownership rather than slogans. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate ETL vs ELT and transformation boundaries by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for ETL vs ELT and transformation boundaries with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-02.

DE09-02 - Raw, staging, clean/intermediate and serving contracts

1. Why it matters

Define explicit contracts for raw, staging, intermediate and serving layers. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Define explicit contracts for raw, staging, intermediate and serving layers.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. Do not blur those responsibilities.

5. New words before we use them

Raw preserves received evidence. Staging standardizes types/keys while surfacing bad rows. Intermediate expresses reusable business logic. Serving exposes stable consumer grain.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — dbt Project Structure

Assigned scope: Whole focused lesson — project layers/structure; map to raw → staging → intermediate → serving

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/Al solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=2dYDS4OQbT0))

URL: <https://www.youtube.com/watch?v=2dYDS4OQbT0>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Inspect the RetailPulse SQLite tables and lineage.json; state grain and allowed mutations at each layer. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to define explicit contracts for raw, staging, intermediate and serving layers and show a concrete result/state that another person can inspect. Observable target: For Raw, staging, clean/intermediate and serving contracts: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The workflow is trustworthy because the received input is identified and preserved before transformation, so every downstream result can be traced back to unchanged source evidence. Your independent check is: Do not validate Raw, staging, clean/intermediate and serving contracts by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If layers blur together, ask: can I reconstruct source evidence; where is invalid data visible; what table is safe for consumers?

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. 3. Run the earliest independent control: Do not validate Raw, staging, clean/intermediate and serving contracts by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write the four layer contracts before writing transformations.

13. Validate your result

Do not validate Raw, staging, clean/intermediate and serving contracts by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Raw, staging, clean/intermediate and serving contracts in your own words. Start with the purpose: Define explicit contracts for raw, staging, intermediate and serving layers. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Raw, staging, clean/intermediate and serving contracts by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Raw, staging, clean/intermediate and serving contracts with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-03.

DE09-03 - Deterministic transformations and production SQL style

1. Why it matters

Write deterministic transformations whose output is a function of declared inputs and rules. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Write deterministic transformations whose output is a function of declared inputs and rules.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

A deterministic transformation is a function: same declared input state -> same output. Production SQL makes grain, keys, filters and ordering explicit so reruns do not depend on accidents.

5. New words before we use them

Deterministic work avoids hidden manual edits, unstable ordering and ambient state. Production SQL should make grain, filters, joins and calculations readable.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — dbt Models

Assigned scope: Whole focused lesson — deterministic SQL models and dependencies

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=JQYz-8sl1aQ)

URL: <https://www.youtube.com/watch?v=JQYz-8sl1aQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run the baseline twice from a fresh database and compare serving controls. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to write deterministic transformations whose output is a function of declared inputs and rules and show a concrete result/state that another person can inspect. Observable target: For Deterministic transformations and production SQL style: Run an independent control query for row count, key coverage and one important total; inspect a deliberately changed filter/key case to prove the query logic rather than memorization.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: Do not validate Deterministic transformations and production SQL style by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If reruns differ, isolate nondeterminism: time-dependent defaults, unordered tie selection, mutable inputs or hidden state.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A deterministic transformation is a function: same declared input state -> same output. 3. Run the earliest independent control: Do not validate Deterministic transformations and production SQL style by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Rewrite one ambiguous transformation as explicit typed/filter logic and add an independent control query.

13. Validate your result

Do not validate Deterministic transformations and production SQL style by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Deterministic transformations and production SQL style in your own words. Start with the purpose: Write deterministic transformations whose output is a function of declared inputs and rules. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Deterministic transformations and production SQL style by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Deterministic transformations and production SQL style with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-04.

DE09-04 - Deduplication and key integrity

1. Why it matters

Separate exact duplicate removal from business-key conflict resolution. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Separate exact duplicate removal from business-key conflict resolution.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Deduplication is not “drop duplicates”; it is choosing the one valid survivor for an explicitly defined business key and tie-breaker while proving what was removed.

5. New words before we use them

An exact duplicate repeats the same received row. A repeated business key with changed attributes is not automatically a duplicate; choose a version/merge rule and prove it.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — dbt Tests

Assigned scope: Focused tests class — uniqueness/not-null/relationship thinking; book adds dedup key-integrity workflow

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=bvZ-rJm7uMU)

URL: <https://www.youtube.com/watch?v=bvZ-rJm7uMU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

RetailPulse contains one exact R10 duplicate. The pipeline quarantines it while preserving source evidence. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to separate exact duplicate removal from business-key conflict resolution and show a concrete result/state that another person can inspect. Observable target: For Deduplication and key integrity: Record counts before/after, list the exact failing keys/rows, and prove rejected/duplicate/missing cases are explained rather than silently dropped.

9. Why it worked

This works because separate exact duplicate removal from business-key conflict resolution is made explicit before the result is accepted. Your independent check is: Do not validate Deduplication and key integrity by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

Never add DISTINCT to silence unexpected fan-out. Identify the key/grain first.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Deduplication is not “drop duplicates”; it is choosing the one valid survivor for an explicitly defined business key and tie-breaker while proving what was removed. 3. Run the earliest independent control: Do not validate Deduplication and key integrity by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Prove raw received rows, exact duplicate count and one-current-row order_id in staging/serving.

13. Validate your result

Do not validate Deduplication and key integrity by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Deduplication and key integrity in your own words. Start with the purpose: Separate exact duplicate removal from business-key conflict resolution. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Deduplication and key integrity by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Deduplication and key integrity with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-05.

DE09-05 - Schema checks and data contracts

1. Why it matters

Enforce a schema/data contract and quarantine incompatible rows without silently coercing them. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Enforce a schema/data contract and quarantine incompatible rows without silently coercing them.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

A data contract is an agreement about fields, types, nullability, keys and semantics. Schema checks are the automated doorway that blocks incompatible data before it spreads.

5. New words before we use them

A contract includes required fields, types, domain rules and evolution expectations. Compatibility decisions should be explicit.

6. Useful video / source

Source: Confluent — How to use Data Contracts for Long-Term Schema Management

Assigned scope: 0:00–17:00: data-contract purpose, JSON-blob problems, Avro/Protobuf advantages; book/lab adds batch schema checks and quarantine

How to use it: Watch the assigned contract/schema section, then implement the course schema checks and failure handling. The streaming examples are a concept bridge, not a replacement for the batch contract lab.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ZlJB8cs-cJU)

URL: <https://www.youtube.com/watch?v=ZlJB8cs-cJU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

R11 has bad date and R12 misses customer_id; both remain visible in quarantine. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to enforce a schema/data contract and quarantine incompatible rows without silently coercing them and show a concrete result/state that another person can inspect. Observable target: For Schema checks and data contracts: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: Do not validate Schema checks and data contracts by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If a cast fails, preserve the raw value and reason; do not convert every error to zero/null.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A data contract is an agreement about fields, types, nullability, keys and semantics. 3. Run the earliest independent control: Do not validate Schema checks and data contracts by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Add one safe optional-field evolution proposal and one breaking-change example.

13. Validate your result

Do not validate Schema checks and data contracts by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Schema checks and data contracts in your own words. Start with the purpose: Enforce a schema/data contract and quarantine incompatible rows without silently coercing them. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Schema checks and data contracts by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Schema checks and data contracts with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-06.

DE09-06 - Source-to-target reconciliation

1. Why it matters

Reconcile source, staging and serving with row and value controls. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Reconcile source, staging and serving with row and value controls.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Reconciliation is like closing a cash register: independently compare row counts, keys, and important totals to the source before trusting a cleaned or summarized result.

5. New words before we use them

Counts alone can pass while money is wrong; sums alone can pass while rows are duplicated. Reconciliation needs both plus exclusion rules.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — dbt Sources

Assigned scope: Sources/freshness context; pair with our source-to-target reconciliation lab

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=7CrrXazV_8k))

URL: https://www.youtube.com/watch?v=7CrrXazV_8k

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Baseline: 14 received, 11 accepted unique, 3 quarantined, 10 completed serving rows, revenue 2990.00. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to reconcile source, staging and serving with row and value controls and show a concrete result/state that another person can inspect. Observable target: For Source-to-target reconciliation: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because reconcile source, staging and serving with row and value controls is made explicit before the result is accepted. Your independent check is: Do not validate Source-to-target reconciliation by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If totals disagree, compare layer by layer until the first divergence; repair there.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Reconciliation is like closing a cash register: independently compare row counts, keys, and important totals to the source before trusting a cleaned or summarized result. 3. Run the earliest independent control: Do not validate Source-to-target reconciliation by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write controls for source rows, accepted rows, quarantined rows, serving rows, serving revenue and Cancelled exclusion.

13. Validate your result

Do not validate Source-to-target reconciliation by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Source-to-target reconciliation in your own words. Start with the purpose: Reconcile source, staging and serving with row and value controls. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Source-to-target reconciliation by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Source-to-target reconciliation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-07.

DE09-07 - Incremental processing and safe reruns

1. Why it matters

Process new/changed records incrementally and make reruns safe. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Process new/changed records incrementally and make reruns safe.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Incremental processing handles only the new/changed slice, while safe reruns guarantee replay of the same slice does not create duplicate or divergent results.

5. New words before we use them

Incremental models need a trustworthy cursor and unique key. Reprocessing the same batch must not create duplicates.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — incremental loading; book/lab proves safe rerun and watermark/state behavior

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5eMytPBgmVs)

URL: <https://www.youtube.com/watch?v=5eMytPBgmVs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Apply R6 update + R14 new record; serving becomes 11 rows / 3490.00. Rerun selects 0. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to process new/changed records incrementally and make reruns safe and show a concrete result/state that another person can inspect. Observable target: For Incremental processing and safe reruns: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: Do not validate Incremental processing and safe reruns by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If a changed key duplicates, check unique-key logic and $\geq$ cursor boundary before changing data.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Incremental processing handles only the new/changed slice, while safe reruns guarantee replay of the same slice does not create duplicate or divergent results. 3. Run the earliest independent control: Do not validate Incremental processing and safe reruns by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Explain why `updated_at` is the cursor and `order_id` is the merge key; show rerun evidence.

13. Validate your result

Do not validate Incremental processing and safe reruns by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Incremental processing and safe reruns in your own words. Start with the purpose: Process new/changed records incrementally and make reruns safe. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Incremental processing and safe reruns by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Incremental processing and safe reruns with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-08.

DE09-08 - Failure behavior, retries, quarantine and atomic state

1. Why it matters

Keep destination and checkpoint/state atomic when a batch fails validation. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Keep destination and checkpoint/state atomic when a batch fails validation.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Schema drift is the input contract changing under you. Compatible changes can be handled deliberately; incompatible rows belong in quarantine with the exact reason and original payload.

5. New words before we use them

State must advance only after the destination commit is durable. Retry transient work; quarantine record-level defects; fail the run for contract-breaking conditions according to policy.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — dbt Commands

Assigned scope: Focused commands/build class; course failure lab adds retry/quarantine/atomic-state behavior

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=t4OeWHW3SsA))

URL: <https://www.youtube.com/watch?v=t4OeWHW3SsA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Failure batch contains a numeric defect. `fail_on_quarantine=True` returns `FAILED_VALIDATION` and leaves watermark unchanged. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to keep destination and checkpoint/state atomic when a batch fails validation and show a concrete result/state that another person can inspect. Observable target: For Failure behavior, retries, quarantine and atomic state: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because keep destination and checkpoint/state atomic when a batch fails validation is made explicit before the result is accepted. Your independent check is: Do not validate Failure behavior, retries, quarantine and atomic state by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If state moved on failure, treat it as a correctness bug, not a logging issue.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Schema drift is the input contract changing under you. 3. Run the earliest independent control: Do not validate Failure behavior, retries, quarantine and atomic state by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Inject one failure and prove destination row count and state are unchanged.

13. Validate your result

Do not validate Failure behavior, retries, quarantine and atomic state by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Failure behavior, retries, quarantine and atomic state in your own words. Start with the purpose: Keep destination and checkpoint/state atomic when a batch fails validation. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Failure behavior, retries, quarantine and atomic state by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Failure behavior, retries, quarantine and atomic state with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-09.

DE09-09 - Lineage, run metadata and auditability

1. Why it matters

Produce lineage and run metadata sufficient to explain what ran and what changed. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Produce lineage and run metadata sufficient to explain what ran and what changed.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Observability is the pipeline dashboard: how many records arrived, how late, which run processed them, what failed, and how long each stage took.

5. New words before we use them

Auditability links source file, run id, layer, counts, state before/after, status and output controls. Lineage records dependencies and grains.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — dbt Documentation

Assigned scope: Whole focused lesson — documentation/lineage context

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=UqoWyMjcqrA)

URL: <https://www.youtube.com/watch?v=UqoWyMjcqrA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Inspect run_audit plus lineage.json produced by the pipeline. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to produce lineage and run metadata sufficient to explain what ran and what changed and show a concrete result/state that another person can inspect. Observable target: For Lineage, run metadata and auditability: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because produce lineage and run metadata sufficient to explain what ran and what changed is made explicit before the result is accepted. Your independent check is: Do not validate Lineage, run metadata and auditability by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If you cannot answer which input/run produced a row, the handoff is incomplete.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Observability is the pipeline dashboard: how many records arrived, how late, which run processed them, what failed, and how long each stage took. 3. Run the earliest independent control: Do not validate Lineage, run metadata and auditability by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Trace R6 from source update through staging to serving and identify its run metadata.

13. Validate your result

Do not validate Lineage, run metadata and auditability by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Lineage, run metadata and auditability in your own words. Start with the purpose: Produce lineage and run metadata sufficient to explain what ran and what changed. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Lineage, run metadata and auditability by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Lineage, run metadata and auditability with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-10.

DE09-10 - Build and defend a reliable batch pipeline

1. Why it matters

Build and defend a reliable batch pipeline end to end. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Build and defend a reliable batch pipeline end to end.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence.

5. New words before we use them

A reliable batch path is not just transformation code: contracts -> validation -> quarantine -> deterministic transform -> reconciliation -> tests -> atomic state -> metadata -> rerun proof.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp — dbt video course sequence

Assigned scope: Main-class review: project structure → sources → models → tests → docs; then close video and build independently

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=2dYDS4OQbT0)

URL: <https://www.youtube.com/watch?v=2dYDS4OQbT0>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run `m09_pipeline.py` from a clean output folder and read `M09_SMOKE_RESULT.json`. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to build and defend a reliable batch pipeline end to end and show a concrete result/state that another person can inspect. Observable target: For Build and defend a reliable batch pipeline: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because build and defend a reliable batch pipeline end to end is made explicit before the result is accepted. Your independent check is: Do not validate Build and defend a reliable batch pipeline by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

When a result is plausible but controls fail, trust controls and investigate.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence. 3. Run the earliest independent control: Do not validate Build and defend a reliable batch pipeline by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Rebuild the pipeline on a changed file and produce an evidence packet, not only a success message.

13. Validate your result

Do not validate Build and defend a reliable batch pipeline by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Build and defend a reliable batch pipeline in your own words. Start with the purpose: Build and defend a reliable batch pipeline end to end. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Build and defend a reliable batch pipeline by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Build and defend a reliable batch pipeline with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-11.

DE09-11 - dbt-style project anatomy, sources and dependency graph

1. Why it matters

Read a dbt-style project as a dependency graph instead of disconnected SQL files. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Read a dbt-style project as a dependency graph instead of disconnected SQL files.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Picture dbt as a directed graph of models: sources feed transformations, refs express dependencies, tests guard contracts, and materializations decide how results are stored.

5. New words before we use them

`source()` represents declared external/source relations; `ref()` creates model dependencies; project/YAML config adds metadata/tests/materialization behavior.

6. Useful video / source

Source: DataTalksClub Zoomcamp — dbt project structure

Assigned scope: Whole focused lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=2dYDS4OQbT0)

URL: <https://www.youtube.com/watch?v=2dYDS4OQbT0>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Open `dbt_style_project`: `source -> stg_orders -> int_completed_orders -> fct_sales`. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to read a dbt-style project as a dependency graph instead of disconnected SQL files and show a concrete result/state that another person can inspect. Observable target: For dbt-style project anatomy, sources and dependency graph: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because read a dbt-style project as a dependency graph instead of disconnected SQL files is made explicit before the result is accepted. Your independent check is: Do not validate dbt-style project anatomy, sources and dependency graph by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If the graph is unclear, follow source()/ref() edges before reading every SQL expression.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture dbt as a directed graph of models: sources feed transformations, refs express dependencies, tests guard contracts, and materializations decide how results are stored. 3. Run the earliest independent control: Do not validate dbt-style project anatomy, sources and dependency graph by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Sketch the DAG, identify each model grain and explain why direct raw -> mart logic is harder to govern.

13. Validate your result

Do not validate dbt-style project anatomy, sources and dependency graph by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain dbt-style project anatomy, sources and dependency graph in your own words. Start with the purpose: Read a dbt-style project as a dependency graph instead of disconnected SQL files. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate dbt-style project anatomy, sources and dependency graph by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for dbt-style project anatomy, sources and dependency graph with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-12.

DE09-12 - Executable data tests, quality severity and CI boundaries

1. Why it matters

Turn quality expectations into executable tests with severity and CI boundaries. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Turn quality expectations into executable tests with severity and CI boundaries.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

A data test turns an expectation into executable evidence. Severity decides whether a failure blocks publication, warns, or routes repair; CI runs those checks before changes are promoted.

5. New words before we use them

Blocking tests protect invariants such as required keys/uniqueness. Warning tests surface risk that may not justify stopping every run. CI should fail only on deliberately blocking conditions.

6. Useful video / source

Source: DataTalksClub Zoomcamp — dbt tests

Assigned scope: Whole focused lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=bvZ-rJm7uMU))

URL: <https://www.youtube.com/watch?v=bvZ-rJm7uMU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

The SQLite smoke runs unique key, required customer, positive revenue and reconciliation tests. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to turn quality expectations into executable tests with severity and CI boundaries and show a concrete result/state that another person can inspect. Observable target: For Executable data tests, quality severity and CI boundaries: Record counts before/after, list the exact failing keys/rows, and prove rejected/duplicate/missing cases are explained rather than silently dropped.

9. Why it worked

The spreadsheet result is trustworthy because the source structure/formula logic is preserved and an independent cell/row/total check proves the displayed result came from the intended calculation. Your independent check is: Do not validate Executable data tests, quality severity and CI boundaries by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

Do not make every anomaly fatal and do not make critical integrity checks warnings.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A data test turns an expectation into executable evidence. 3. Run the earliest independent control: Do not validate Executable data tests, quality severity and CI boundaries by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Classify five proposed checks as BLOCK or WARN and defend the business consequence.

13. Validate your result

Do not validate Executable data tests, quality severity and CI boundaries by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Executable data tests, quality severity and CI boundaries in your own words. Start with the purpose: Turn quality expectations into executable tests with severity and CI boundaries. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Executable data tests, quality severity and CI boundaries by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Executable data tests, quality severity and CI boundaries with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE09-13.

DE09-13 - Incremental models, materialization, documentation and handoff

1. Why it matters

Explain materialization/incremental behavior and hand off a reproducible project. This matters because a transformation pipeline can publish plausible but wrong data unless layer contracts, rerun behavior and reconciliation are explicit.

2. What you are learning

Explain materialization/incremental behavior and hand off a reproducible project.

3. Prerequisite / what you already know

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

4. Picture it in your head

Treat schema change like a controlled migration: record before-state, apply versioned DDL, verify data/contracts, and prove you can restore or rebuild from documented artifacts.

5. New words before we use them

Views, tables and incremental models trade freshness, compute and persistence. Current dbt incremental models require valid SQL for both full/incremental paths; unique_key supports updates instead of blind append.

6. Useful video / source

Source: DataTalksClub Zoomcamp — dbt models + commands

Assigned scope: Whole focused lesson; then use dbt commands lesson if needed

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=JQYz-8sl1aQ)

URL: <https://www.youtube.com/watch?v=JQYz-8sl1aQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Inspect fct_sales.sql with materialized=incremental, unique_key=order_id and on_schema_change=fail; compare it with the executed SQLite equivalent. Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

8. Expected result / what you should see

You should be able to explain materialization/incremental behavior and hand off a reproducible project and show a concrete result/state that another person can inspect. Observable target: For Incremental models, materialization, documentation and handoff: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because explain materialization/incremental behavior and hand off a reproducible project is made explicit before the result is accepted. Your independent check is: Do not validate Incremental models, materialization, documentation and handoff by appearance alone. The proof is deterministic source-to-target reconciliation and explicit failure behavior.

10. Common beginner mistakes

If incremental logic changes historical meaning, plan a controlled rebuild/backfill rather than assuming future runs repair history.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Treat schema change like a controlled migration: record before-state, apply versioned DDL, verify data/contracts, and prove you can restore or rebuild from documented artifacts. 3. Run the earliest independent control: Do not validate Incremental models, materialization, documentation and handoff by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a handoff: inputs, grain, model DAG, tests, state/cursor, full-refresh/backfill decision, known limitations and rerun commands.

13. Validate your result

Do not validate Incremental models, materialization, documentation and handoff by appearance alone. Use the lesson-specific control: Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Incremental models, materialization, documentation and handoff in your own words. Start with the purpose: Explain materialization/incremental behavior and hand off a reproducible project. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Incremental models, materialization, documentation and handoff by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Incremental models, materialization, documentation and handoff with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.